MODUL TUGAS PRAKTIKUM
CRUD DENGAN LARAVEL 13

Pertemuan 1–7 + Persiapan UTS

Nama: Yustina Fauwawan
NIM: 2511039
Program Studi: Teknologi Rekayasa Internet

Tahun Akademik 2026/2027

Pertemuan 1 — Instalasi, MVC, Route dan View

Tugas: Kumpulkan screenshot halaman profil-kelas, hasil php artisan --version, route:list, dan dua commit Git.

Langkah kerja

1. Verifikasi php, Composer, Node/NPM, dan Git.

2. Buat/siapkan proyek Laravel 13.

3. Buat route /profil-kelas dan view-nya.

4. Tampilkan nama, NIM, dan tujuan belajar.

5. Jalankan route:list dan buat dua commit Git.

Kode

routes/web.php

```
Route::view('/profil-kelas', 'profil-kelas')->name('profil-kelas');
```

resources/views/profil-kelas.blade.php

```
<h1>Kelas Laravel 13</h1>
<p>Nama: Yustina Fauwawan</p>
<p>NIM: 2511039</p>
<p>Tujuan: belajar Laravel dan CRUD.</p>
```

PowerShell — Pertemuan 1

```
> php artisan --version
Laravel Framework 13.x.x

> php artisan route:list --path=profil
GET  profil-kelas  ...  profil-kelas
```

http://localhost:8000/profil-kelas

profil-kelas

Kelas Laravel 13

Nama: Yustina Fauwawan

NIM: 2511039

Tujuan: belajar Laravel dan CRUD

Pertemuan 2 — Routing, Controller dan Blade

Tugas: Buat halaman daftar data film statis menggunakan Route → Controller → Blade; minimal id, judul, sutradara, dan tahun.

Langkah kerja

1. Buat FilmController.

2. Buat route /films dan /films/{id}.

3. Kirim array film dari controller ke Blade.

4. Tampilkan dengan @foreach dan named route.

5. Uji daftar dan detail film.

```
use App\Http\Controllers\FilmController;
use Illuminate\Support\Facades\Route;

Route::get('/films', [FilmController::class, 'index'])->name('films.index');
Route::get('/films/{id}', [FilmController::class, 'show'])
    ->whereNumber('id')->name('films.show');

class FilmController extends Controller
{
    public function index()
    {
        $films = [
            ['id'=>1,'title'=>'Laskar Pelangi','director'=>'Riri Riza','year'=>2008],
            ['id'=>2,'title'=>'Habibie & Ainun','director'=>'Faozan Rizal','year'=>2012],
            ['id'=>3,'title'=>'Keluarga Cemara','director'=>'Yandy Laurens','year'=>2019],
        ];
        return view('films.index', compact('films'));
    }

    public function show(int $id)
    {
        return view('films.show-static', ['id'=>$id]);
    }
}
```

```
@extends('layouts.app')
@section('title', 'Data Film')
@section('content')
<h1>Data Film</h1>
<ul>
@foreach ($films as $film)
<li>
<a href="{{ route('films.show', $film['id']) }}">{{ $film['title'] }}</a>
- {{ $film['director'] }} - {{ $film['year'] }}
</li>
@endforeach
</ul>
@endsection
```

http://localhost:8000/films

films

Data Film

1. Laskar Pelangi — Riri Riza — 2008
2. Habibie & Ainun — Faozan Rizal — 2012
3. Keluarga Cemara — Yandy Laurens — 2019

Pertemuan 3 — Migration, Model, Factory dan Seeder

Tugas: Tambahkan kolom location yang nullable melalui migration baru dan dokumentasikan up()/down(), migrate, serta rollback.

Langkah kerja

1. Buat migration baru untuk location.

2. Pastikan location nullable.

3. Jalankan migrate dan migrate:status.

4. Uji rollback lalu migrate kembali.

5. Catat hasil up() dan down().

```
Schema::create('films', function (Blueprint $table): void {
    $table->id();
    $table->string('title');
    $table->string('director');
    $table->unsignedSmallInteger('year');
    $table->text('description')->nullable();
    $table->timestamps();
});
```

```
class Film extends Model
{
    use HasFactory;
    protected $fillable = ['title','director','year','description'];
    protected function casts(): array
    {
        return ['year' => 'integer'];
    }
}
```

```
Film::factory()->count(10)->create();
```



Pertemuan 4 — Read, Resource Route dan Binding

Tugas: Terapkan fitur Read pada entitas film menggunakan migration, model, seeder, index, dan show.

Langkah kerja

1. Gunakan Route::resource.

2. Ambil data dengan paginate(10).

3. Tampilkan index.

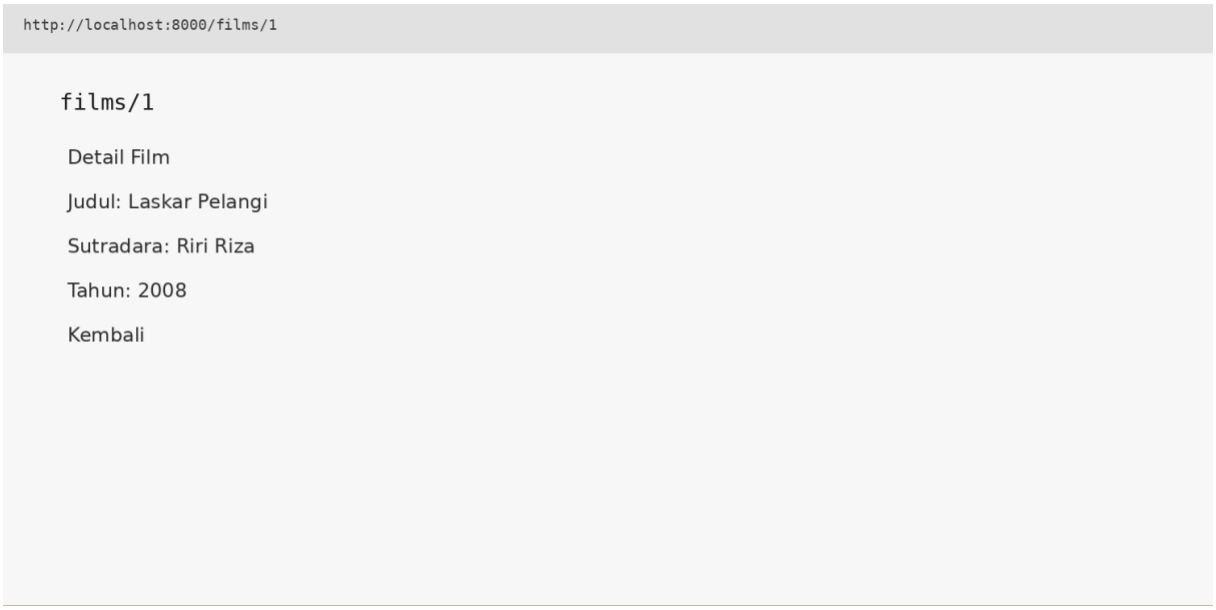
4. Gunakan show(Film $film).

5. Uji ID valid dan ID tidak ada (404).

```
Route::resource('films', FilmController::class);

public function index(): View
{
    $films = Film::query()->latest()->paginate(10);
    return view('films.index', compact('films'));
}

public function show(Film $film): View
{
    return view('films.show', compact('film'));
}
```



Pertemuan 5 — Create, Form, Validasi dan CSRF

Tugas: Lengkapi form create buku sampai seluruh field berfungsi dan sertakan enam kasus uji manual.

Langkah kerja

1. Buat form dengan @csrf.

2. Validasi di server.

3. Gunakan old() dan @error.

4. Simpan hanya data tervalidasi.

5. Uji valid, judul kosong, ISBN duplikat, harga negatif, stok desimal, tahun masa depan.

```
<form method="POST" action="{{ route('films.store') }}">
   @csrf
   <input name="title" value="{{ old('title') }}">
   <input name="director" value="{{ old('director') }}">
   <input name="year" value="{{ old('year') }}">
   @error('title') <small>{{ $message }}</small> @enderror
   <button type="submit">Simpan</button>
</form>
```

```
public function store(StoreFilmRequest $request): RedirectResponse
{
   Film::create($request->validated());
   return redirect()->route('films.index')
      ->with('success', 'Data film berhasil ditambahkan.');
}
```

Kasus uji: valid; judul kosong; ISBN duplikat; harga negatif; stok desimal; tahun masa depan.



Pertemuan 6 — Update, Delete dan Form Reusable

Tugas: Terapkan Update dan Delete pada entitas film, lima uji regresi, dan satu commit khusus refactor form.

Langkah kerja

1. Buat partial _form.blade.php.

2. Gunakan @method('PUT').

3. Gunakan @method('DELETE').

4. Tambahkan konfirmasi hapus.

5. Uji update, validasi, delete, dan URL record terhapus (404).

```
<form method="POST" action="{{ route('films.update', $film) }}">
    @csrf
    @method('PUT')
    <input name="title" value="{{ old('title', $film->title) }}">
    <input name="director" value="{{ old('director', $film->director) }}">
    <input name="year" value="{{ old('year', $film->year) }}">
    <button type="submit">Update</button>
</form>

<form method="POST" action="{{ route('films.destroy', $film) }}">
    @csrf
    @method('DELETE')
    <button type="submit">Hapus</button>
</form>
```

Uji regresi: daftar, detail, update valid, delete, dan URL record terhapus → 404.



Pertemuan 7 — Refaktor, Search, Pagination dan Testing

Tugas: Finalisasi repository Katalog Buku: enam feature test lulus, README instalasi, dan tag Git v1-uts-ready.

Langkah kerja

1. Buat StoreFilmRequest dan UpdateFilmRequest.

2. Pindahkan validasi ke Form Request.

3. Tambahkan search + pagination dengan query string.

4. Jalankan php artisan test --testsuite=Feature --stop-on-failure.

5. Pastikan enam test lulus, README tersedia, lalu buat tag v1-uts-ready.

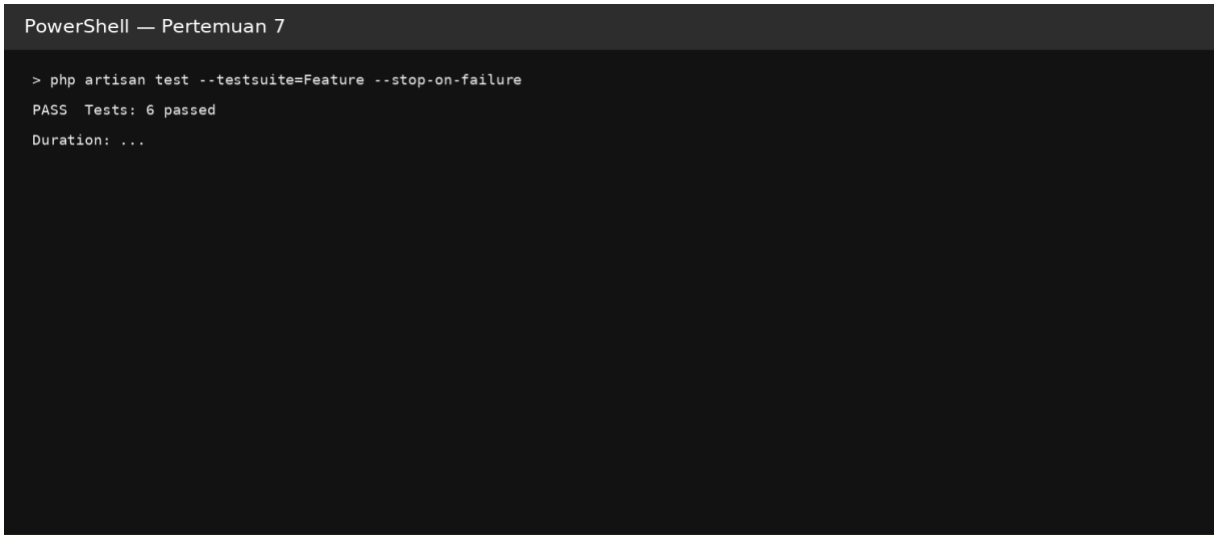
```
class StoreFilmRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        return [
            'title' => ['required','string','max:255'],
            'director' => ['required','string','max:150'],
            'year' => ['required','integer','min:1900','max:'.date('Y')],
            'description' => ['nullable','string'],
        ];
    }
}
```

```
$films = Film::query()
    ->when($request->filled('q'), function ($query) use ($request) {
        $q = $request->string('q');
        $query->where('title', 'like', "%{$q}%")
              ->orWhere('director', 'like', "%{$q}%");
    })
    ->latest()
    ->paginate(10)
    ->withQueryString();
```

```
it('can create film', function () {
    $response = $this->post(route('films.store'), [
        'title'=>'Laskar Pelangi','director'=>'Riri Riza','year'=>2008,
    ]);
    $response->assertRedirect(route('films.index'));
    $this->assertDatabaseHas('films', ['title'=>'Laskar Pelangi']);
});

it('rejects invalid film input', function () {
    $response = $this->post(route('films.store'), [
        'title'=>'','director'=>'','year'=>3000,
    ]);
    $response->assertSessionHasErrors(['title','director','year']);
});
```


PowerShell — Pertemuan 7
> php artisan test --testsuite=Feature --stop-on-failure
PASS Tests: 6 passed
Duration: ...

Persiapan UTS — Inventaris Laboratorium

Target: CRUD inventaris dengan code, name, category, quantity, condition, location, purchase_date, notes; search, pagination, validasi, update, delete, dan feature test.

☐ route:list

☐ halaman index

☐ validasi error

☐ php artisan test

☐ README

☐ tag Git

Sumber: Modul Praktikum CRUD dengan Laravel 13, Versi 1.1, 2026.